



HELWAN NATIONAL UNIVERSITY

Robotics and mechatronics department

Subject: Power electronics

Course Code: POW1310

Title: variable DC power supply from 0V to 24V.

Student Name: Yousef Ahmed Rady Abdelhady Elmetwaly Elbeltagy

Student ID: 922230111

TO: Dr. Eng. Abdullah Mohamed

Table of Contents

1. Introduction
2. Project Objectives
3. Project Overview
4. Components Used
5. Circuit Design Description
 - 5.1 AC to AC Conversion
 - 5.2 AC to DC Conversion
 - 5.3 Arduino-Based PWM Generation and Gate Driving
 - 5.4 Buck Converter and Output Regulation
 - 5.5 LCD Monitoring and User Interface
6. Observations and Challenges
7. Conclusion

Variable DC power supply from 0V to 24V.

This project focuses on designing and implementing a variable DC power supply capable of delivering a regulated output voltage between 0V and 24V. The system utilizes an Arduino microcontroller to generate a PWM signal, which drives a buck converter using an N-channel MOSFET. A potentiometer allows manual control of the output voltage, and an LCD screen provides real-time display of the duty cycle and estimated output voltage. This project combines concepts from power electronics, embedded systems, and real-time control.

1. Project Objectives

Design and implement a variable DC power supply capable of delivering an adjustable output voltage ranging from 0V to 24V.

Use an Arduino UNO to read analog input from a potentiometer and generate a corresponding PWM signal.

Construct a buck converter using an IRFZ44N N-channel MOSFET for efficient voltage regulation.

Drive the MOSFET with a TLP-350 opto-isolated gate driver to ensure safe and effective switching.

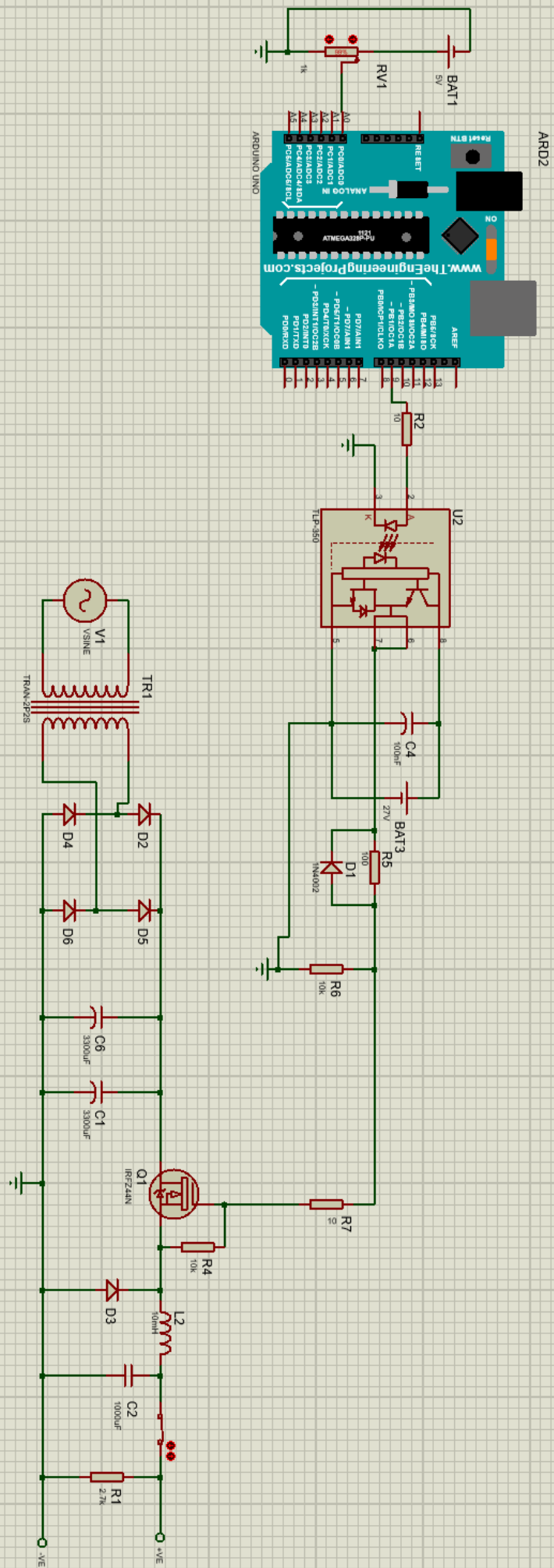
Display real-time data such as PWM duty cycle and estimated output voltage on a 16x2 I2C LCD screen.

Identify and address any nonlinearities or limitations in voltage control performance observed during practical testing.

2. Project Overview

This project is divided into three major parts:

- **AC to AC Conversion using a transformer**
- **AC to DC Conversion using a Full-Wave Bridge Rectifier with Filtering**
- **PWM-Controlled Buck Converter using Arduino and Opto-Isolated Gate Driver**



3. Components Used

The following components were used in the construction of the variable DC power supply:

Component	Specification / Model	Purpose
Arduino UNO	–	Reads potentiometer input and generates PWM output
Potentiometer	1k Ω	User input for adjusting the output voltage
TLP-350	Opto-Isolated Gate Driver	Boosts and isolates the PWM signal for driving the MOSFET
IRFZ44N MOSFET	N-Channel Power MOSFET	Acts as a switching element in the buck converter
LCD Display	16x2 with I2C module	Displays PWM duty cycle and estimated output voltage
Transformer	220V to 23V AC	Steps down AC mains voltage for rectification
Bridge Rectifier	4 $\times$ 1N5408 Diodes	Converts AC to pulsating DC
Filter Capacitors	3300 μ F and 1000 μ F Electrolytic	Smooth the output of the rectifier and the buck converter
Inductor	10mH	Smooths the current in the buck converter
Freewheeling Diode	Ultra-Fast Recovery Diode	Maintains current flow when MOSFET is off
Resistors	2.7K Ω , Pull-down, etc.	Current limiting and signal stabilization
Power Source	Supply 220V AC	Supplies the system with high voltage AC input

5. Circuit Design Description

Part 1: AC to AC Conversion

The first stage of the system handles the conversion of high-voltage AC from the mains (220V) into a lower, safer AC voltage suitable for further processing.

- **Step-Down Transformer**

- A **step-down transformer** is used to reduce the **220V AC** input from the mains to approximately **24V AC RMS** on the secondary winding.
- This is essential to ensure safety and compatibility with the low-voltage electronics and components in the circuit.
- The transformer operates on the principle of **electromagnetic induction**, transferring energy between coils via a magnetic field.

- **Transformer Equation**

The relationship between the primary and secondary voltages is given by the transformer turns ratio:

$$V_p / V_s = N_p / N_s$$

Where:

- V_p = Primary voltage (220V)
- V_s = Secondary voltage (~24V)
- N_p, N_s = Number of turns in the primary and secondary coils

- **Measured Output**

- When tested, the transformer outputs approximately 24V RMS on the secondary winding.
- This AC voltage is then passed to the full-wave rectifier in the next stage.

Part 2: AC to DC Conversion

After the transformer steps down the AC voltage to a safer level (~24V RMS), this voltage must be converted into a DC voltage suitable for use in the buck converter. This is done using a full-wave rectifier followed by filtering capacitors.

a) Full-Wave Bridge Rectifier

- A full-wave bridge rectifier is constructed using four diodes.

- This configuration allows both halves of the AC waveform to be converted into a unidirectional (positive) voltage.
- During each half-cycle:
 - Two diodes conduct and direct the current through the load in the same direction.
- The output of this stage is a pulsating DC waveform.

b) Output Voltage After Rectification

- The peak voltage after rectification is calculated as:

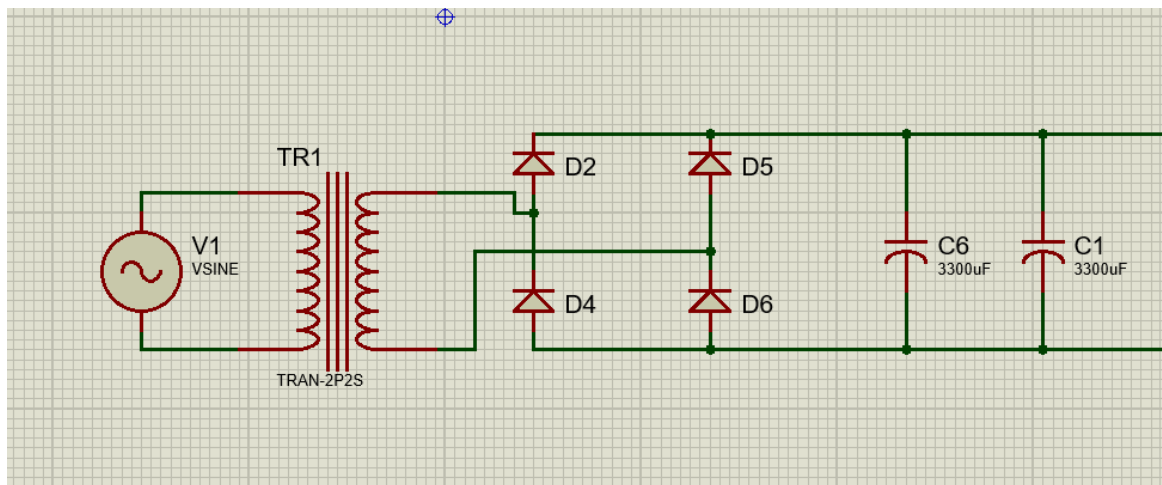
$$V_{\text{peak}} = V_{\text{RMS}} \times \sqrt{2} \approx 24\text{V} \times 1.414 \approx 33.94\text{V}$$

- However, due to the voltage drop across two conducting diodes (each about 0.7V), the effective output becomes:

$$V_{\text{DC(no load)}} = V_{\text{peak}} - (2 \times 0.7\text{V}) \approx 32.54\text{V}$$

c) Filtering with Capacitors

- Two electrolytic capacitors (3300 μF) are placed at the output to filter the pulsating DC signal.
- These capacitors charge up to the peak voltage and discharge slowly, reducing voltage ripple between AC peaks.



Result of This Stage

- The output of this stage is a ~32.5V DC supply with minimal ripple, which is then used as the input to the buck converter for voltage regulation.

Part 3: Arduino-Based PWM Generation and Gate Driving

This part of the system is responsible for generating a pulse-width modulated (PWM) signal that controls the buck converter, thereby adjusting the output voltage based on user input.

a) Potentiometer as User Input

- A 1k Ω potentiometer is used to allow the user to control the desired output voltage.
- It acts as a variable voltage divider connected to the analog input pin A0 of the Arduino UNO.
- The voltage at the potentiometer's wiper ranges from 0V to 5V depending on the knob position.

Analog Reading:

`analogRead(A0)` → Value from 0 to 1023

b) PWM Signal Generation by Arduino

- The Arduino reads the potentiometer value and maps it to a PWM output signal using the `analogWrite()` function.
- The PWM signal is output on **digital pin 9**, which operates at a default frequency of ~490Hz.

PWM Mapping:

$$\text{PWM Value} = \text{map}(\text{analogRead}(A0), 0, 1023, 0, 255)$$
$$\text{Duty Cycle} = (\text{PWM Value}/255) \times 100 \%$$

This PWM duty cycle controls how long the MOSFET stays ON during each cycle, which directly affects the output voltage of the buck converter.

c) Gate Driver – TLP-350

The **TLP-350** is an opto-isolated gate driver used to safely and effectively switch power MOSFETs. It acts as a bridge between the low-voltage PWM signal from the Arduino and the high-power switching side of the circuit.

Purpose of the TLP-350

1. **Electrical Isolation:** The opto-isolation ensures that the high-voltage side (MOSFET and buck converter) is electrically isolated from the low-voltage control side (Arduino), protecting the microcontroller.
2. **Voltage Boosting:** The Arduino only outputs 5V PWM, which is insufficient to fully switch on a power MOSFET like the IRFZ44N. The TLP-350 provides a boosted output suitable for turning on the MOSFET completely and quickly.

How the TLP-350 Circuit Works

1. Input Side (Control Side)

- PWM signal from Arduino pin 9 is passed through a current-limiting resistor (100Ω) to the input LED of the TLP-350 (pins 1 and 2).
- When the PWM signal is HIGH, the internal LED turns on, activating the phototransistor inside the TLP-350.
- This input side is optically isolated from the output side.

2. Output Side (Gate Drive Side)

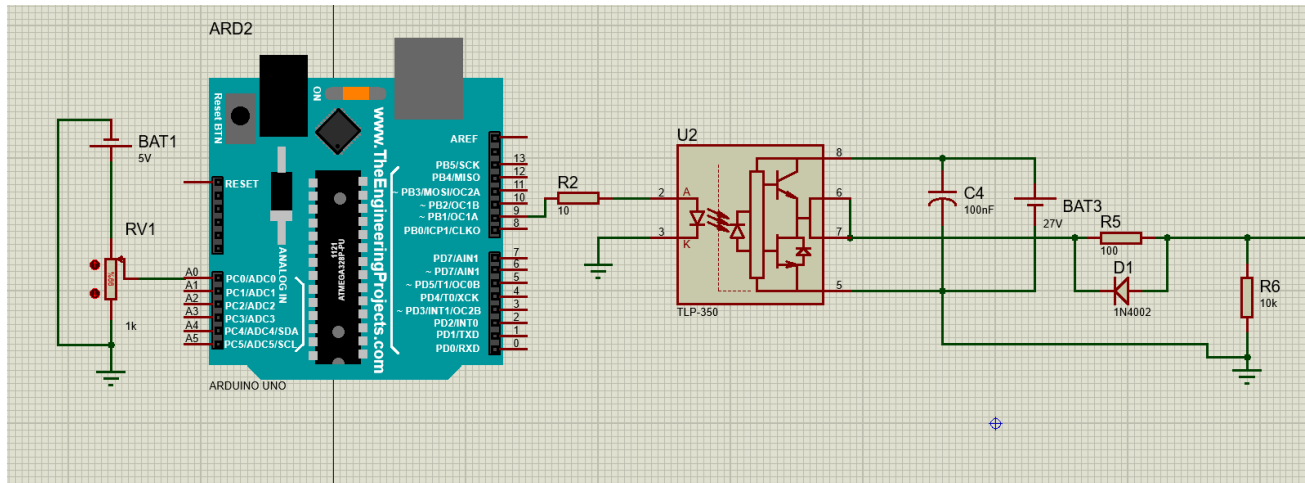
- Powered by an external 18V or 27V DC source (connected to Vcc on pin 8).
- The output (pin 6) drives the gate of the MOSFET.
- Pin 5 is connected to ground.
- Pin 7 is often used as a reference and sometimes for feedback.

When the internal transistor switches on (based on the LED activation), it pulls the gate HIGH, turning the MOSFET ON. When the PWM signal is LOW, the gate is pulled LOW, turning the MOSFET OFF.

Support Components Explained

Component	Role
C4 (100nF)	Bypass capacitor near Vcc to reduce switching noise.
R5 (100Ω)	Limits inrush current to protect the TLP-350.

Component	Role
D1	Protects against reverse polarity from the Vcc supply.
R6 (10k Ω)	Pull-down resistor that discharges the gate to avoid false switching.



Result of This Stage

- The output of this stage is a clean, isolated PWM signal with the appropriate voltage and current capability to switch the IRFZ44N MOSFET effectively.
- This forms the input control signal to the next stage: the buck converter.

Part 4: Buck Converter and Output Regulation

The buck converter section in this circuit is responsible for stepping down the ~33V DC from the rectifier stage to a user-controlled output voltage between 0V and 24V. The control is achieved through PWM signals generated by the Arduino and applied to the gate of the IRFZ44N MOSFET via the TLP-350 driver.

a) Key Components and Their Roles

Component	Value/Type	Role
Q1	IRFZ44N	Power MOSFET that acts as the main switch
R4	10k Ω	Pull-down resistor to keep the gate LOW when PWM is OFF
L2	10mH	Stores energy when the switch is ON and releases it during OFF periods
D3	Ultra Fast Recovery Diode	Freewheeling diode that maintains current flow during the MOSFET OFF state
C2	1000 μ F	Filters the output and reduces voltage ripple
R1	2.7k Ω	Load resistor used to measure and test output voltage

b) How It Works

PWM Control of the MOSFET (Q1)

- The PWM signal from the Arduino is applied to Q1 through the TLP-350 gate driver.
- When the PWM is HIGH, Q1 turns ON and allows current to flow from the input supply through the inductor to the output and load.
- When the PWM is LOW, Q1 turns OFF, and the inductor L2 continues to deliver current to the load by releasing its stored energy through D3 (freewheeling diode).

Inductor (L2) Function

- During the ON time: stores energy.
- During the OFF time: discharges energy, smoothing current flow and preventing sudden voltage drops.

Freewheeling Diode (D3)

- Prevents backflow and voltage spikes when the MOSFET switches OFF.

- Ensures continuity of inductor current.

Output Capacitor (C2)

- Provides a **low impedance path** for the AC component (ripple) and helps maintain a **steady DC output**.

Load Resistor (R1)

- Allows you to safely test and measure the output voltage.
- Acts as a constant load for simulation and real-world testing.

c) Voltage Regulation Equation

$$V_{\text{out}} = D \times V_{\text{in}}$$

Where:

- D is the duty cycle of the PWM (0–1)
- V_{in} is the rectified DC input (~31V)
- V_{out} is the desired output (0V to 24V based on pot control)

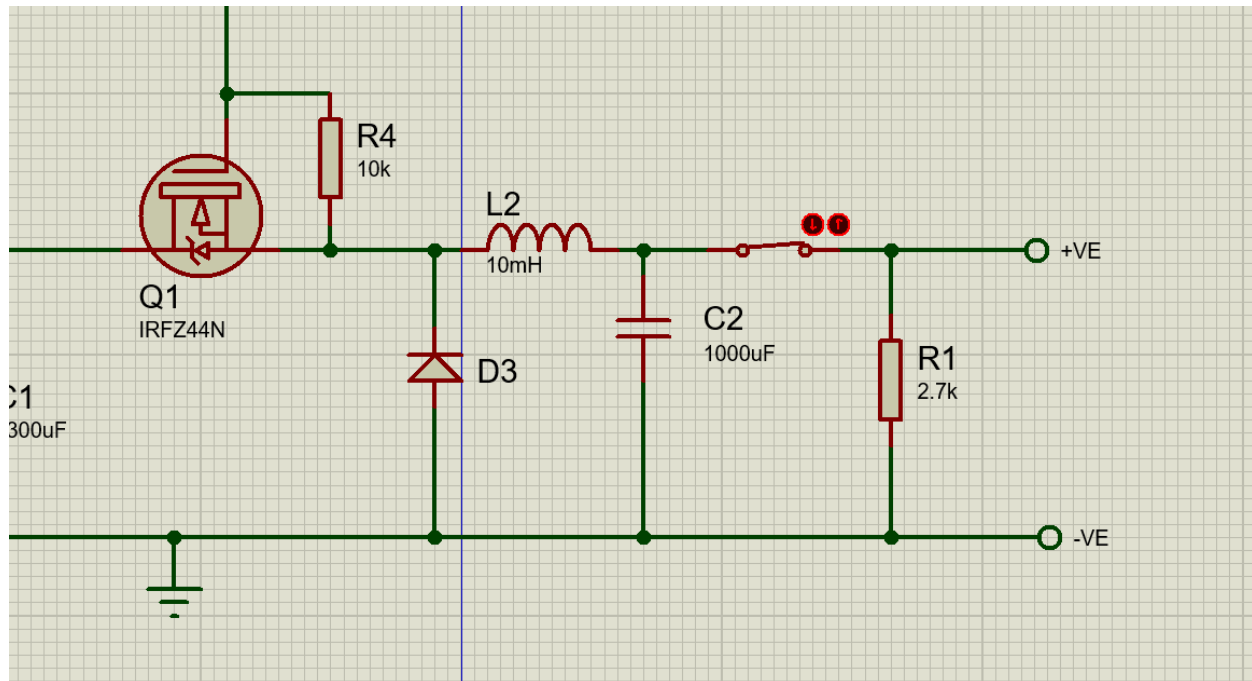
For example, at 80% duty cycle:

$$V_{\text{out}} \approx 0.8 \times 31\text{V} \approx 24.8\text{V}$$

d) Practical Notes

- At low duty cycles, voltage changes are very sensitive due to the inductor response.
- At higher duty cycles, the voltage increases slowly toward the max.

This explains the nonlinear voltage behavior you observed in real-life testing.



Result of This Stage

The output of the buck converter is a smooth and stable DC voltage, which can be varied by the user using a potentiometer. This voltage is suitable for powering other circuits, modules, or small loads and is monitored in real-time using the LCD display.

Part 6: LCD Monitoring and User Interface

To make the system more user-friendly and informative, a 16x2 LCD screen with an I2C interface is integrated into the design. It allows real-time display of the PWM duty cycle and the estimated output voltage based on the current duty cycle.

a) Purpose of the LCD Display

- Provide real-time feedback to the user about the system's operating conditions.
- Display two key values:
 1. **PWM Duty Cycle (%)**
 2. **Estimated Output Voltage (V)**

This makes it easier to observe how turning the potentiometer affects the output without needing a multimeter.

b) Connection and Communication

- The LCD uses the I2C protocol, which reduces the number of required Arduino pins to just two:
 - SDA (Data) → A4 on Arduino
 - SCL (Clock) → A5 on Arduino
- The LiquidCrystal_I2C library is used to interface with the display.

c) Software Implementation

In the Arduino code:

- The potentiometer is read using `analogRead(A0)`.
- The value is mapped to a PWM range using `map()` and written to PWM pin 9.
- The duty cycle and estimated voltage are then calculated and displayed on the LCD.

Example Calculation:

```
int pwmValue = map(analogRead(A0), 0, 1023, 0, 255);  
float dutyPercent = pwmValue / 255.0 * 100.0;  
float outputVoltage = dutyPercent / 100.0 * 31.0; // assuming 31V input
```

Display Output:

- Line 1: Duty: 45%
- Line 2: Volt: 13.9V

Benefits of Adding LCD Monitoring

- No need for external measurement tools.
- Enhances understanding of PWM-to-voltage relationship.
- Useful for debugging or adjusting load requirements in real time.

Observations and Challenges

During the testing and practical implementation of the variable DC power supply, several behaviors were observed both expected and unexpected. This section highlights key findings and issues encountered:

1. Nonlinear Output Voltage Behavior

- **Observation:** The output voltage did not increase uniformly with the potentiometer rotation.
 - In the first quarter turn, the voltage jumped quickly from 0V to around 20V.
 - The remaining rotation only raised the voltage slightly up to 24V.
- **Cause:** This nonlinearity was due to the combined effect of:
 - The large inductor (10mH) delaying current rise.
 - The default low PWM frequency (~490 Hz).
 - The mathematical relation between duty cycle and voltage not being truly linear in practical scenarios.

2. Low PWM Frequency from Arduino

- **Issue:** The default frequency on Arduino's pin 9 (~490 Hz) resulted in:
 - Audible noise from the inductor.
 - Ripple at low voltages.
 - Slower response and less smooth voltage transitions.
- **Planned Solution:** Consider increasing the PWM frequency using Timer1 configuration to ~31 kHz for more stable buck operation.

3. High Gate Drive Voltage Required

- **Challenge:** The IRFZ44N MOSFET requires a high gate voltage (~10V–18) to fully turn on.
- **Solution:** Using the **TLP-350 gate driver** solved this by:
 - Providing proper gate voltage.
 - Electrically isolating Arduino from the power stage.

4. Voltage Control Based on Estimation

- **Observation:** The voltage shown on the LCD was calculated based on PWM duty cycle, not directly measured.
- **Limitation:** This means it may not reflect exact voltage under all load conditions.
- **Future Enhancement:** Add a voltage divider to sense the actual output voltage and display accurate real-time readings on the LCD.

5. Circuit Worked Effectively with Light Loads

- **Success:** With a load resistor of $2.7\text{k}\Omega$, the circuit provided stable and adjustable output.
- **Limitation:** The circuit has not yet been tested with higher power loads such as motors or LEDs.

Conclusion

The design and implementation of the Arduino-based variable DC power supply using a buck converter was successfully achieved. The system was able to convert an unregulated 31V DC input into a smooth and adjustable output voltage ranging from 0V to 24V. Control was achieved through a potentiometer, with real-time monitoring provided by an I2C-connected 16x2 LCD display.

The use of a TLP-350 opto-isolated gate driver proved to be an effective solution for safely and efficiently driving the IRFZ44N power MOSFET, ensuring robust switching performance. Additionally, the buck converter stage demonstrated consistent performance for low to moderate load conditions.

Although the system exhibited some nonlinear behavior in voltage control, this was understood to be a result of hardware limitations and the characteristics of the PWM control method. These issues provide valuable insight and a foundation for further improvements in future versions of the project.

Overall, the project successfully demonstrated key principles of power electronics, embedded control using Arduino, and real-time feedback mechanisms, making it a valuable learning experience and a functional prototype for applications requiring variable DC output.